BSc (Hons) in Computer Science & Artificial Intelligence

Course Work 1: Technical Report: Design and Evaluate Neural Networks for SMS Spam Detection





Project Title: SMS Spam Detection with Neural Networks



- Student Name : W.A.Thisumi Nimsani
- Student ID : KIC-BSCCSAI-251P-030

- Module : Neural Network
- Word Count: ~1460 (Excluding equations, tables, references)
- Date of Submission: Nov 10, 2026

Table of Contents

Table of Contents **2**
List of Tables **2**
List of Figures **3**
T1 Problem Definition and Data Preparation **4**
T1(a) Problem and Provenance 4
T1(b) Data Audit 4
T1(c) Leakage-Safe Pipeline 6
T2 Implement and Verify Learning from First Principles **7**
T2(a) Forward Computation 7
T2(b) Backpropagation 7
T2(c) Gradient Verification 8
T2(d) Numerical Diagnosis 8
T3 Develop the Baseline and Framework Models **10**
T3(a) Baseline 10
T3(b) Two Neural Architectures 10
T3(c) Training Control 10
T3(d) Traceable Implementation 11
T4 Results and Fair Comparison **12**
T5 Error Analysis and Robustness **13**
T6 Inference, Deployment and Limitations **14**
References **16**
Appendix **16**

List of Tables

Table 1. Data audit summary..... 6
Table 2. Validation and Test Performance of Final Models Across Random Seeds.. 12

List of Figures

Figure 1. Class distribution of ham and spam messages..... 5
Figure 2. Message length by class: boxplot (top) and histogram (bottom)..... 5
Figure 3. NumPy MLP learning curves (training and validation loss)..... 8
Figure 4. Tiny-batch overfitting check (16 samples)..... 9
Figure 5. PyTorch MLP learning curves across seeds 11, 22, 33..... 11
Figure 6. Text CNN learning curves across seeds 11, 22, 33..... 11
Figure 7. Test precision–recall comparison across the three final models..... 14

T1 Problem Definition and Data Preparation

T1(a) Problem and Provenance

The intended users of this system are mobile network operators and SMS gateway providers, organizations requiring an automated method to protect subscribers from bulk unsolicited messaging. The task is formulated as a binary classification problem: each incoming message is categorized as either genuine (ham, class 0) or spam (class 1). The unit of analysis is a single SMS message. The practical benefits include reducing nuisance traffic, blocking phishing attempts, and safeguarding users from premium-rate scams, while maintaining the integrity of legitimate communication.

The dataset is sourced from the UCI SMS Spam Collection (Almeida, Hidalgo & Yamakami, 2011), a publicly available research corpus. It contains 5,572 messages, comprising 4,825 ham (86.6%) and 747 spam (13.4%). Several ethical considerations are relevant. The corpus predominantly features Singapore and UK English, which limits the generalizability of the findings. Additionally, there is an operational risk that false positives may inadvertently block important personal messages. Regarding personal data, only information present in the publicly released corpus is included.

T1(b) Data Audit

An initial examination of the data revealed no missing values, 403 duplicate rows, and no empty messages. The class distribution is highly imbalanced (Figure 1), indicating that accuracy alone would be a misleading metric; therefore, spam F1 is adopted as the primary evaluation metric. Message length distinctly differentiates the two classes (Figure 2): spam messages average 138.7 characters, while ham messages average 71.5 characters, with spam concentrated around 130–160 characters and ham exhibiting greater variability. Given the strong predictive signal from message length, it is essential to benchmark any model against a length-aware baseline.

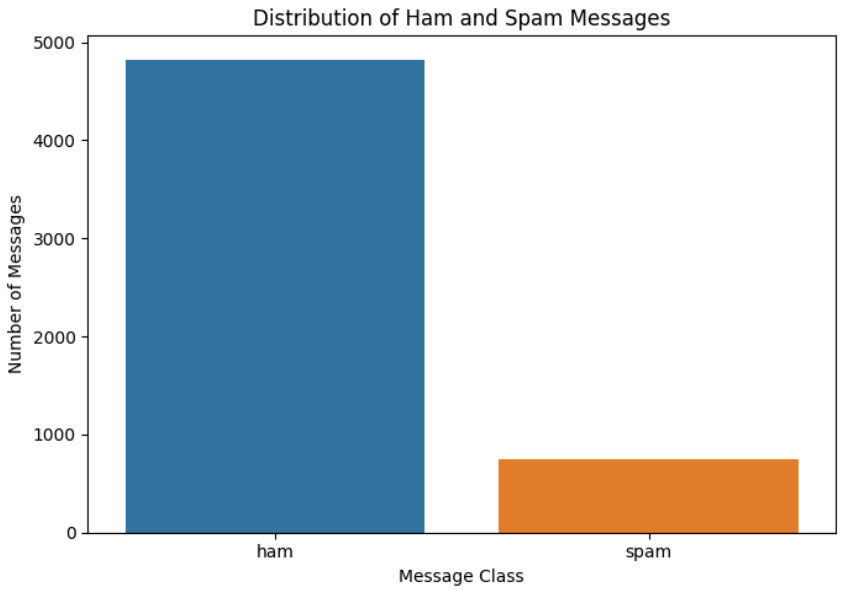


Figure 1. Class distribution of ham and spam messages.

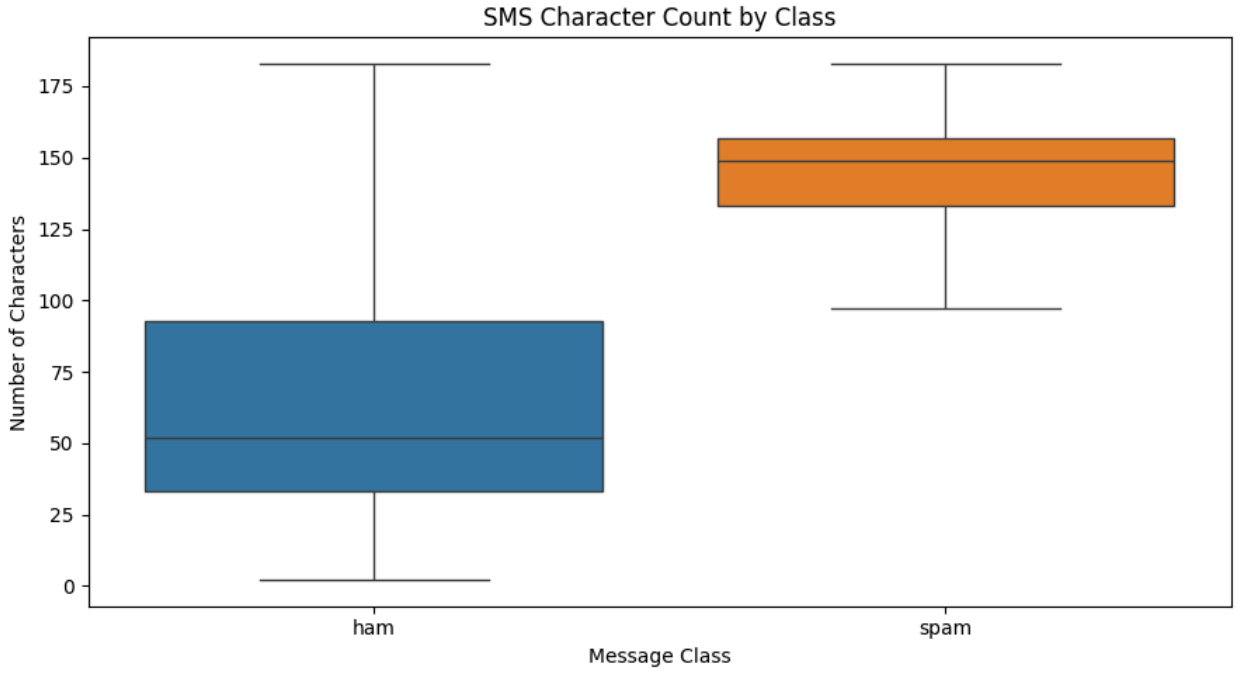


Figure 2. Message length by class: boxplot (top) and histogram (bottom).

Table 1. Data audit summary

Missing Values	0	None Required
Duplicate rows	403	Removed before splitting
Empty messages	0	None required
Class imbalance	86.6% / 13.4%	Spam F1 as primary metric
Length difference	Strong	Length-aware baselines

T1(c) Leakage-Safe Pipeline

Duplicate entries were removed prior to dataset splitting, reducing the corpus to 5,169 messages (4,516 ham, 653 spam). A stratified 70/15/15 split produced 3,617 training, 776 validation, and 776 test messages, maintaining consistent class proportions (87.37% ham, 12.63% spam) across all subsets. The TF-IDF vectorizer was fitted exclusively on the training data and subsequently applied to the validation and test sets, preventing vocabulary leakage. Split indices were stored in CSV files and configuration settings in JSON format. A normalized overlap audit (Step 10) confirmed the absence of exact duplicates across splits, though a small number of case- and whitespace-normalized matches were identified (train–test: 5; train–validation: 2; validation–test: 1). While these represent a minor threat to validity, the more significant issue of exact duplicates is fully mitigated.

T2 Implement and Verify Learning from First Principles

T2(a) Forward Computation

The foundational model is a single-hidden-layer multilayer perceptron (MLP) implemented in NumPy without automatic differentiation, utilizing 32-dimensional TruncatedSVD features derived from the TF-IDF matrix. Weights are initialized with scaled Gaussian noise, the hidden layer employs the tanh activation function, and the output layer uses a sigmoid function with binary cross-entropy loss. The array dimensions are as follows: X (3617, 32); W1 (32, 16); b1 (1, 16); W2 (16, 1); b2 (1, 1). While there is a loose analogy to biological neurons—where W represents synaptic weights, b the firing threshold, and tanh the bounded non-linearity—this comparison is limited, as real neurons exhibit spiking behavior and temporal dynamics rather than continuous activations.

T2(b) Backpropagation

Analytical gradients were computed using the chain rule and applied in mini-batch gradient descent (learning rate 0.1, batch size 64, 100 epochs). Training loss decreased from 0.694 to 0.087, while validation loss stabilized at 0.098 (Figure 3). The loss curve remained stable, with minor overfitting observed after approximately epoch 80.

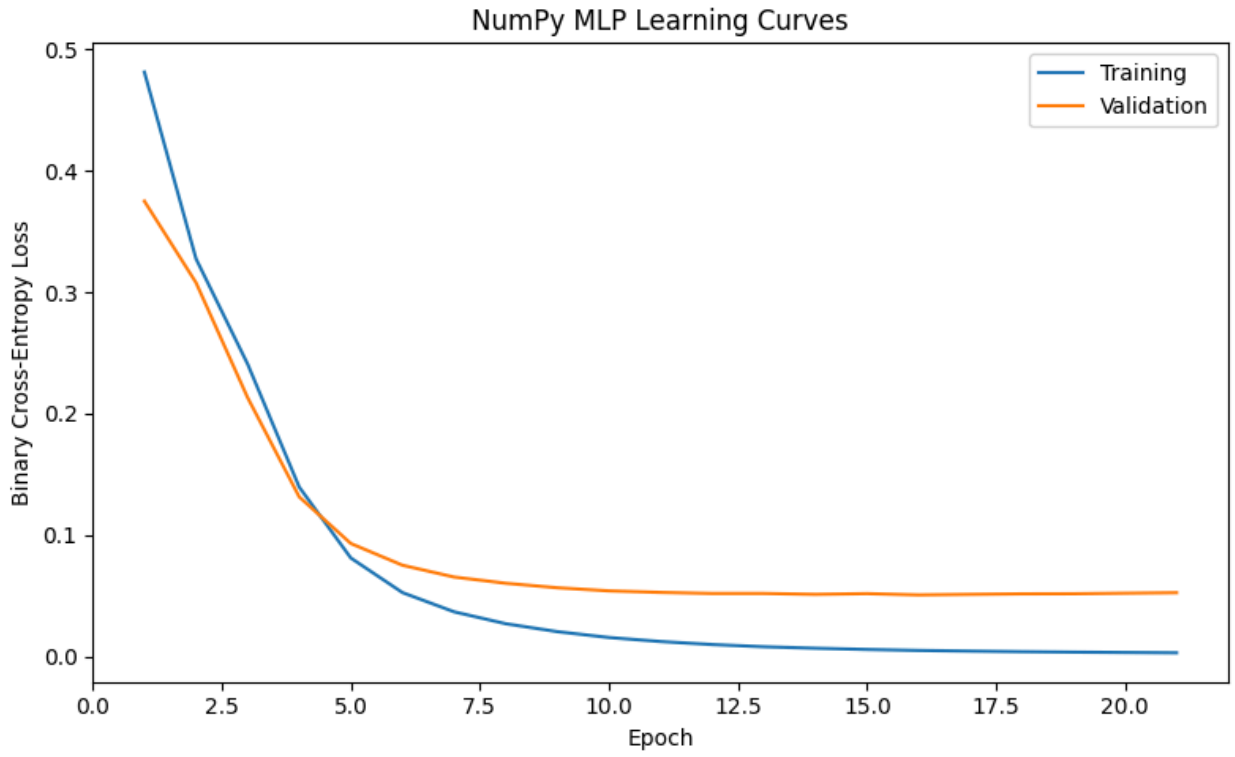


Figure 3. NumPy MLP learning curves (training and validation loss).

T2(c) Gradient Verification

Five parameters, distributed across W1, b1, W2, and b2, were evaluated using central finite differences (ε = 1e-5, batch size 8). All parameters satisfied both criteria (absolute error < 1e-7 and relative error < 1e-4), with the largest absolute error recorded at 7.2e-12. This tolerance is justified by numerical considerations: central differences introduce $O(\varepsilon^2)$ truncation error and $O(1/\varepsilon)$ round-off error, resulting in a numerical floor near 1e-8 at ε = 1e-5. Errors significantly below this threshold provide strong evidence for correct implementation.

T2(d) Numerical Diagnosis

A deliberately small batch of 16 samples (8 ham, 8 spam) was overfitted using a learning rate of 0.3 and a stopping criterion of loss $\leq$ 0.03 or 3,000 updates. The model achieved a loss of 0.0299 in 459 updates (Figure 4). This confirms the correct numerical behaviour of gradients and

updates. Failure to fit this small batch would have indicated issues with gradient computation or learning-rate scaling rather than data quality.

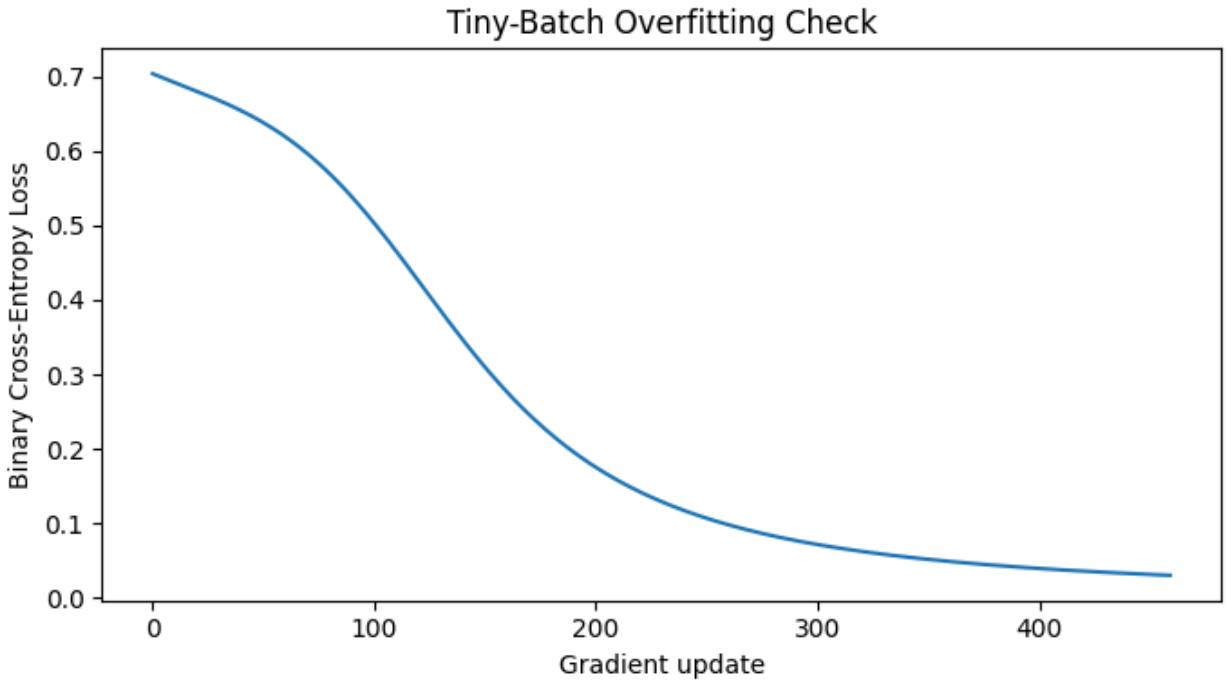


Figure 4. Tiny-batch overfitting check (16 samples).

T3 Develop the Baseline and Framework Models

T3(a) Baseline

Logistic regression (max_iter = 1000, scikit-learn 1.6.1) was trained on the same TF-IDF features. On the validation set, it achieved an accuracy of 0.960, spam precision of 1.000, recall of 0.684, and spam F1 of 0.812. The combination of high precision and low recall indicates a highly conservative baseline: approximately one-third of spam messages are missed, but ham messages are rarely misclassified as spam.

T3(b) Two Neural Architectures

The PyTorch MLP runs 10,000 → 64 → 32 → 1 with ReLU activations, dropout 0.3, and Xavier initialisation (642,177 parameters). The Text CNN uses a 7,232-token embedding (dimension 32), a Conv1D layer with 32 filters and kernel size 3, masked global max pooling, dropout 0.3, and a linear output (234,561 parameters). Layer tables showing input/output shapes, activations, and parameter counts are saved as pytorch_mlp_layers.csv and text_cnn_layers.csv. The MLP is a natural fit for sparse bag-of-words features; the CNN is better suited to short sequential text because it can lean on local word order (Kim, 2014).

T3(c) Training Control

Both neural models were trained with a batch size of 64, the Adam optimizer (learning rate 0.001, weight decay 1e-4), early stopping (patience 5, min_delta 1e-5), and checkpointing at the lowest validation loss. Three random seeds (11, 22, 33) were used for each architecture. Training and validation loss curves are presented in Figures 5 and 6. The MLP converged between epochs 9 and 11, while the CNN converged between epochs 17 and 23, with no observed instability in either model.

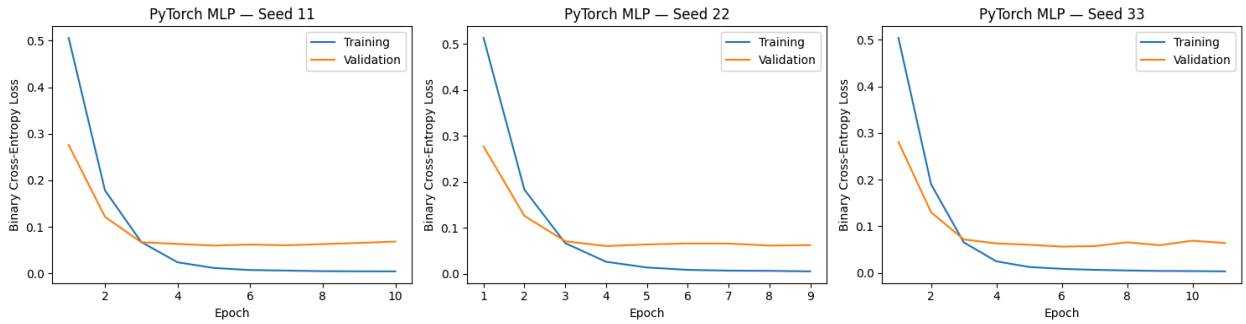


Figure 5. PyTorch MLP learning curves across seeds 11, 22, 33.

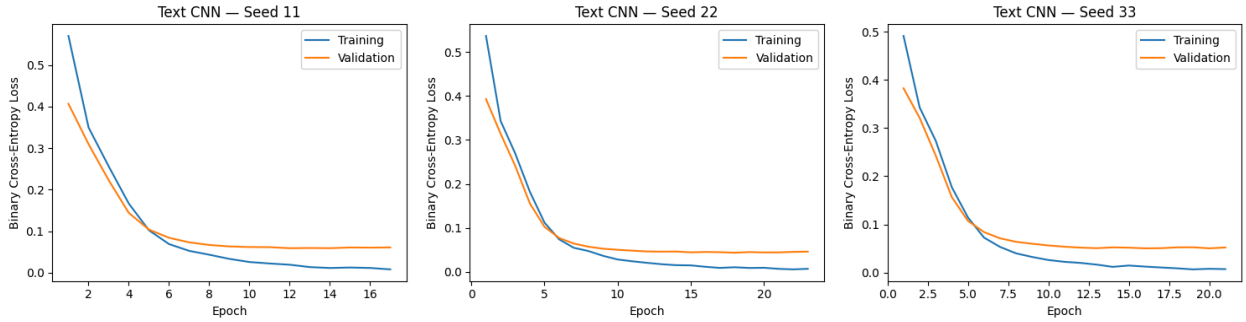


Figure 6. Text CNN learning curves across seeds 11, 22, 33.

T3(d) Traceable Implementation

Preparation, training, and evaluation are organized into separate sections within the notebook, and no pretrained components are utilized. All model code is original. The TF-IDF vectorizer, logistic regression, and SVD are sourced from scikit-learn (Pedregosa et al., 2011), while training-loop utilities (DataLoader, Adam, BCEWithLogitsLoss) are provided by PyTorch (Paszke et al., 2019). Framework versions and the computational environment are documented in pytorch_environment.json. Each reported result is traceable to a saved configuration and checkpoint, as listed in model_results_traceability.csv; for example, pytorch_mlp_run_11.json corresponds to pytorch_mlp_seed_11.pt.

T4 Results and Fair Comparison

The tuning budget was intentionally limited: one predefined configuration per architecture and three random seeds per model, with model selection based solely on validation spam F1. The test set remained unused until final evaluation. A dropout ablation on the CNN (0.3 to 0.0) resulted in a mean validation F1 change of −0.0018, with per-seed variations from +0.0007 to −0.0056 (Table 2). Consequently, dropout has minimal impact at this dataset size, does not degrade performance, and is retained as a regularization measure.

Table 2. Validation and Test Performance of Final Models Across Random Seeds

Results linking model	Configuration	Seed	Validation	Test Spam F1
Logistic Regression	experiment_settings.json	42	0.812	0.840
PyTorch MLP	pytorch_mlp_run_11.json	11	0.925	0.932
PyTorch MLP	pytorch_mlp_run_22.json	22	0.914	0.922
PyTorch MLP	pytorch_mlp_run_33.json	33	0.920	0.932
Text CNN	text_cnn_run_11.json	11	0.937	0.923
Text CNN	text_cnn_run_22.json	22	0.959	0.929
Text CNN	text_cnn_run_33.json	33	0.942	0.916

On the test set, the MLP achieved an average spam F1 of 0.929 (standard deviation 0.006), while the CNN achieved 0.923 (standard deviation 0.007). The validation ranking, which placed the CNN slightly ahead, did not persist in the test results; the performance difference between the two models falls within the range of seed variance. Comprehensive per-model accuracy, precision, and recall metrics are provided in final_test_results.csv.

T5 Error Analysis and Robustness

The representative CNN model produced 15 test errors: 7 false positives and 8 false negatives. False positives were typically short ham messages containing vocabulary similar to spam ("Glad to see your reply", "On the road so cant txt"). False negatives were atypical spam, such as link-style or short-code messages ("ringtoneking 84484"), where the CNN's n-gram features failed to detect a strong spam pattern. Both error types constitute a small proportion of the test set (approximately 1.9%), but false positives have a greater user impact than false negatives. Therefore, borderline cases should be directed to human review rather than being automatically blocked. A stress test removed 10% of non-whitespace characters using a fixed seed (2026), applied uniformly across all models without retraining. The mean test spam F1 score for the CNN decreased by 0.067 (recall −0.105), for the MLP by 0.041 (recall −0.061), and for logistic regression by 0.196 (recall −0.245). Character-level corruption had the greatest impact on the CNN, as its embedding lookup depends on exact token matches. In contrast, the MLP's TF-IDF representation demonstrated greater robustness. These results are particularly relevant for applications involving noisy or auto-corrected text.

T6 Inference, Deployment and Limitations

On a CPU with a single thread (torch.set_num_threads(1); environment documented in inference_benchmark_environment.json), the median latency at batch size 1 was 1.027 ms for logistic regression, 1.178 ms for the MLP, and 0.504 ms for the CNN, including preprocessing. The saved-model demonstration rejects empty input, non-string input, non-ASCII scripts, and inputs exceeding 2,000 characters, returning a structured error instead of misclassifying. Figure 7 presents the precision–recall comparison, indicating that the CNN and MLP perform similarly and both outperform the baseline.

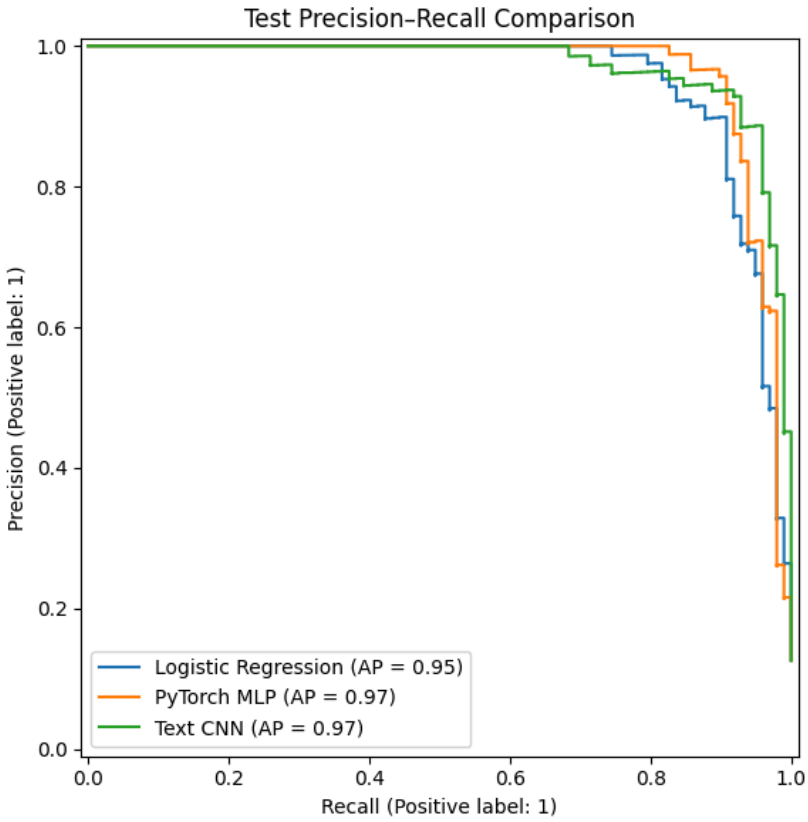


Figure 7. Test precision–recall comparison across the three final models.

Limitations and responsible use. The corpus is restricted to English (Singapore/UK), and the model has not been validated for Sinhala, Tamil, or code-mixed text. The demonstration system blocks non-ASCII scripts to address this limitation. As spam campaigns evolve over time, analyses should be repeated on newly labeled data at least quarterly. The 0.5 threshold serves as a default rather than an optimal value; given the asymmetric costs of false positives and false negatives, the threshold should be adjusted for each deployment. Model scores represent raw sigmoid outputs rather than calibrated probabilities, so Platt scaling or isotonic regression is required before interpreting them as risk estimates (Guo et al., 2017). Additionally, the documented normalized overlap between data splits and the residual risk of shared campaign templates indicate that reported test scores may be slightly optimistic.

Recommendation. The Text CNN is recommended as the primary classifier because it achieves the highest clean-text validation F1 score, the lowest latency, and test performance comparable to the MLP. It should be used in conjunction with a calibration layer and manual review for scores within the 0.3 to 0.7 range. The PyTorch MLP remains as a fallback option due to its greater robustness under character corruption. The NumPy MLP serves solely as a teaching artifact and is not suitable for deployment.

References

Almeida, T. A., Hidalgo, J. M. G., & Yamakami, A. (2011). Contributions to the study of SMS spam filtering: New collection and results. In Proceedings of the 11th ACM Symposium on Document Engineering (pp. 259–262). ACM. https://doi.org/10.1145/2034691.2034742

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. In Proceedings of the 34th International Conference on Machine Learning (Vol. 70, pp. 1321–1330). PMLR.

Kim, Y. (2014). Convolutional neural networks for sentence classification. In Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP) (pp. 1746–1751). Association for Computational Linguistics. https://doi.org/10.3115/v1/D14-1181

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., … Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. In Advances in Neural Information Processing Systems (Vol. 32, pp. 8024–8035).

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., … Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Appendix

GitHub Repository: The source code and project implementation are available at: https://github.com/ThisumiWijesinghe/SMS_SPAM_DETECTION_ANN

Google Drive: The complete project files, datasets, experimental results, and supporting materials are available at: https://drive.google.com/drive/folders/12Y8kfSb89Sias3Dq-CZAqhJrBI6zok1u?usp=sharing